Day 7 - Notes

Apex Unit Tests

What are Apex Unit Tests?

Apex unit tests are used to:

- Test Apex classes and triggers
- Ensure code works correctly
- Verify future updates don't break existing functionality
- Meet Salesforce deployment requirements

Salesforce provides an Apex testing framework to:

- Write tests
- Run tests
- Check code coverage

Importance of Apex Unit Tests

1. Ensures Correct Functionality

Tests verify that Apex code behaves as expected.

---

2. Regression Testing

Tests can be rerun after updates to ensure old functionality still works.

---

3. Required for Deployment

Before deploying Apex code:

- At least 75% code coverage is required

- All tests must pass
- Every trigger must have some coverage

---

4. Produces High-Quality Apps

- Better user experience
- More reliable applications
- Increased customer trust

Test Class Syntax

Test Class

```
@IsTest
private class MyTestClass {

}
```

Test Method

```
@IsTest
static void myTestMethod() {

}
```

Assertions in Apex

Assertions verify expected results.

Common Assertions

Assert.areEqual()

```
Assert.areEqual(expected, actual);
```

Assert.isTrue()

```
Assert.isTrue(condition);
```

Assert.isFalse()

```
Assert.isFalse(condition);
```

Assert.isNull()

```
Assert.isNull(value);
```

Example: Temperature Converter

Apex Class

```
public class TemperatureConverter {

    public static Decimal FahrenheitToCelsius(Decimal fh) {

        Decimal cs = (fh - 32) * 5/9;
        return cs.setScale(2);
    }
}
```

Test Class

```
@IsTest
private class TemperatureConverterTest {

    @IsTest
    static void testWarmTemp() {

        Decimal celsius =
            TemperatureConverter.FahrenheitToCelsius(70);

        Assert.areEqual(21.11, celsius);
    }
}
```

Code Coverage

Definition

Percentage of Apex code executed during tests.

Deployment Requirement

Minimum:

```
75%
```

Best Practice

Aim for:

```
100% coverage
```

Causes of Low Coverage

1. Missing Conditional Branches

Example:

```
if(condition) {

}
else {

}
```

Both branches must be tested.

---

2. Missing Boundary Values

Test:

- Positive values
- Negative values
- Null values
- Maximum/minimum values

TaskUtil Example

Apex Class

```
public class TaskUtil {

    public static String getTaskPriority(String leadState) {

        if(String.isBlank(leadState)
            || leadState.length() > 2) {

            return null;
        }

        if(leadState == 'CA') {
```

```
            return 'High';
        }
        else {
            return 'Normal';
        }
    }
}
```

Test Methods Needed

1. Normal Case

```
'NY' → 'Normal'
```

---

2. High Priority Case

```
'CA' → 'High'
```

---

3. Invalid Input

```
'Montana' → null
```

Test Suites

Definition

Collection of test classes run together.

Uses

- Deployment testing
- Regression testing
- Organized testing

Test Data in Apex

Important Point

Test records:

- Are NOT permanently stored
- Roll back after test completion

SeeAllData

Syntax

```
@IsTest(SeeAllData=true)
```

Purpose

Allows access to existing org data.

Best Practice

Avoid using it unless absolutely necessary.

Apex Trigger Testing

Trigger Example: AccountDeletion

Purpose

Prevent deleting accounts with related opportunities.

Trigger

```
trigger AccountDeletion on Account (before delete) {

    for(Account a :
        [SELECT Id FROM Account
         WHERE Id IN
         (SELECT AccountId FROM Opportunity)
         AND Id IN :Trigger.old]) {

        Trigger.oldMap.get(a.Id)
            .addError(
            'Cannot delete account with related opportunities.');
    }
}
```

Trigger Test Class

Steps

1. Create test data
2. Perform DML operation
3. Verify results

Example

```
@IsTest
private class TestAccountDeletion {

    @IsTest
    static void testDelete() {
```

```
        Account acct =
            new Account(Name='Test');

        insert acct;

        Opportunity opp =
            new Opportunity(
                Name='Opp',
                StageName='Prospecting',
                CloseDate=System.today(),
                AccountId=acct.Id
            );

        insert opp;

        Database.DeleteResult result =
            Database.delete(acct, false);

        Assert.isFalse(result.isSuccess());
    }
}
```

Test.startTest() and Test.stopTest()

Purpose

Creates fresh governor limits for test execution.

Syntax

```
Test.startTest();

/* code */

Test.stopTest();
```

Trigger Test Cases

Valid Insert

Should succeed.

Invalid Insert

Should throw error.

Invalid Update

Should throw error.

Test Utility Classes

Purpose

Reusable test data creation.

Advantages

- Reduces repeated code
- Easier maintenance
- Cleaner test methods

TestDataFactory Example

```
@IsTest
public class TestDataFactory {

    public static List<Account>
```

```
    createAccountsWithOpps(
        Integer numAccts,
        Integer numOppsPerAcct) {

        List<Account> accts =
            new List<Account>();

        return accts;
    }
}
```

Benefits

- Creates multiple records quickly
- Reusable across tests
- Supports bulk testing

Bulk Testing

Why Important?

Triggers execute on collections of records.

Best Practice

Test:

- Single record
- Multiple records (bulk)

Important Rule

DO NOT use:

```
@IsTest
```

Asynchronous Processing Basics

Definition

Runs processes in the background without making users wait.

Synchronous vs Asynchronous

Synchronous	Asynchronous
User waits	Runs in background
Sequential	Parallel/background

Benefits

1. User Efficiency

Users continue working while processing happens.

---

2. Scalability

Supports more processing using parallel execution.

---

3. Higher Governor Limits

Feature	Sync	Async
SOQL Queries	100	200

Types of Asynchronous Apex

Type	Use
Future Methods	Callouts
Batch Apex	Large data
Queueable Apex	Chained jobs
Scheduled Apex	Timed execution

Future Methods

- Background processing
- External callouts

Batch Apex

- Handles huge datasets
- Data cleanup/archiving

Queueable Apex

- Supports job chaining
- Allows complex data types

Scheduled Apex

Runs jobs:

- Daily
- Weekly

- At specific times

Async Processing Lifecycle

1. Enqueue

Request added to queue.

---

2. Persistence

Stored safely.

---

3. Dequeue

Request processed.

Resource Conservation

Salesforce reduces async processing if:

- CPU usage high
- Memory usage high
  to protect system stability.

Important Exam Points

Minimum Deployment Coverage

```
75%
```

Best Coverage

```
100%
```

Future Method Use

```
Web service callouts
```

Batch Apex Use

```
Processing large datasets
```

Queueable Advantage

```
Job chaining
```

Scheduled Apex Use

```
Timed execution
```

Salesforce DX Environment Setup Notes

Introduction to Salesforce DX

Salesforce Developer Experience (DX) is a set of tools that helps developers:

- Streamline the application development lifecycle
- Improve team collaboration

- Enable automated testing and CI/CD
- Manage source-driven development
- Create temporary development environments called Scratch Orgs

---

Important Concepts

What is a Scratch Org?

A Scratch Org is:

- A temporary Salesforce org
- Fully configurable
- Used for development and testing
- Created quickly using Salesforce CLI
- Deleted after work is completed

Uses of Scratch Org

- Feature development
- Testing
- Team collaboration
- Isolated environments

What is a Dev Hub?

A Dev Hub is the main Salesforce org used to:

- Create scratch orgs
- Manage scratch orgs
- Authenticate Salesforce DX projects

Important

You must enable Dev Hub in:

- Trailhead Playground OR
- Developer Edition Org

Salesforce CLI Setup

Install Salesforce CLI

Download Salesforce CLI from:

https://developer.salesforce.com/tools/salesforcecli

Verify Installation

Run:

```
sf --version
```

Example output:

```
@salesforce/cli/2.134.6 win32-x64 node-v22.22.2
```

Update CLI

```
sf update
```

Enable Dev Hub

Steps

1. Open Salesforce Org
2. Click Setup
3. Search for:

```
Dev Hub
```

4. Enable:

- Dev Hub
- Optional: Second Generation Packaging

4. Save

Login to Dev Hub using Salesforce CLI

Command

```
sf org login web -d -a DevHub
```

Explanation

- `-d` → sets as default Dev Hub
- `-a DevHub` → creates alias named DevHub

What Happens?

- Salesforce login page opens
- Login using:
  - Trailhead Playground credentials OR
  - Developer Edition org credentials

Successful Output

```
Successfully authorized username@example.com
```

Using VS Code

Recommended Terminal

Use only one terminal:

- VS Code Terminal (recommended)
- OR PowerShell
- OR Command Prompt

Create Salesforce Project

Open VS Code Terminal

```
Terminal → New Terminal
```

Create Project Folder

```
mkdir my_sf_project
cd my_sf_project
```

Install Git

If Git is not installed:

Download from:

https://git-scm.com/download/win

Verify installation:

```
git --version
```

Clone Dreamhouse Project

Clone Repository

```
git clone https://github.com/trailheadapps/dreamhouse-lwc.git
```

Open Project Folder

```
cd dreamhouse-lwc
```

Create Git Branch

Command

```
git checkout -b my_branch
```

Output

```
Switched to a new branch 'my_branch'
```

Salesforce CLI Help

Command

```
sf --help
```

Shows all available Salesforce CLI commands.

Create Scratch Org

Command

```
sf org create scratch -d -f config/project-scratch-def.json -a dreamhouse-org
```

Explanation

- `-d` → sets scratch org as default
- `-f` → uses scratch org configuration file
- `-a dreamhouse-org` → creates alias

Successful Output Example

```
Your scratch org is ready.
```

Example details:

```
OrgId: 00Dcf00000Awsv7
Username: test-d2tolheaaaep@example.com
Alias: dreamhouse-org
```

Warning Message

```
Record types defined in the scratch org definition file will stop being
capitalized by default in a future release.
```

This warning can be ignored.

Open Scratch Org

Command

```
sf org open
```

What Happens?

- Browser opens automatically
- Logs into scratch org using saved authentication

Deploy Project to Scratch Org

Command

```
sf project deploy start
```

Purpose

Deploys all metadata from local project to scratch org.

Assign Permission Set

Command

```
sf org assign permset -n Dreamhouse
```

Purpose

Provides access to Dreamhouse application.

Import Sample Data

Command

```
sf data import tree -p data/sample-data-plan.json
```

Purpose

Imports sample data into scratch org.

Test Dreamhouse Application

Open Scratch Org Again

```
sf org open
```

Open Dreamhouse App

Steps

1. Open App Launcher (9 dots)
2. Search:

```
Dreamhouse
```

3. Open app

Verify Data

Properties Tab

Should contain approximately:

```
12 properties
```

Brokers Tab

Should contain approximately:

```
8 brokers
```

Important Salesforce CLI Commands

Command	Purpose
`sf --version`	Check CLI version
`sf update`	Update CLI
`sf org login web -d -a DevHub`	Login to Dev Hub
`sf org create scratch -d -f config/project-scratch-def.json -a dreamhouse-org`	Create scratch org
`sf org open`	Open org in browser

Command	Purpose
`sf project deploy start`	Deploy metadata
`sf org assign permset -n Dreamhouse`	Assign permission set
`sf data import tree -p data/sample-data-plan.json`	Import sample data
`sf --help`	Show CLI commands

Common Errors and Solutions

Error

```
'sf' is not recognized
```

Solution

- Install Salesforce CLI
- Restart terminal
- Run:

```
sf --version
```

Key Learnings:

- Apex tests are used to test classes and triggers.
- Minimum 75% code coverage is needed for deployment.
- All tests must pass before deployment.
- `@IsTest` is used for test classes.
- Assertions check expected results.
- Test both valid and invalid cases.
- Test data is automatically deleted after tests finish.

- `Test.startTest()` and `Test.stopTest()` give fresh limits.
- Triggers should always have test classes.
- Bulk testing is important for triggers.
- Test utility classes help reuse test data code.
- Asynchronous Apex runs in the background.
- Async processing improves performance and user efficiency.
- Async Apex gets higher governor limits.
- Types of async Apex:
  - Future Methods
  - Batch Apex
  - Queueable Apex
  - Scheduled Apex
- Future methods are used for callouts.
- Batch Apex is used for large data processing.
- Queueable Apex supports chaining jobs.
- Scheduled Apex runs jobs at fixed times.
- Salesforce uses queues to process async jobs.
- Queueable Apex is preferred over future methods.
- Learned about Salesforce DX
- Understood Dev Hub and Scratch Org
- Installed and used Salesforce CLI
- Logged into Salesforce org using CLI
- Learned difference between Trailhead and Salesforce org accounts
- Used VS Code for Salesforce development
- Installed and used Git
- Cloned project from GitHub
- Created and managed Git branches
- Created a Scratch Org
- Opened Scratch Org using CLI
- Deployed project metadata to Scratch Org
- Assigned Permission Sets
- Imported sample data into Scratch Org
- Tested Dreamhouse application
- Used important Salesforce CLI commands
- Learned basic troubleshooting for CLI and Git errors

- Understood Salesforce DX development workflow